Real-Time Multi-Core Sorting and Median Filter Service

A Report

Submitted for the course

26ELC331 Real Time Operating System using QNX

by

Anmol Govindarajapuram Krishnan (CB.EN.U4EEE23103)

Mauli Rajguru (CB.EN.U4EEE23120)

Sanjay TS (CB.EN.U4EEE23136)



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 17, 2026

Contents

List of Figures iii

List of Tables iv

Abbreviations v

List of Symbols vi

1 Introduction 1

1.1 Background and Motivation 1

1.2 Project Objectives 1

1.3 Workload and Dataset 2

1.4 Report Structure 3

2 System Overview 4

2.1 Introduction 4

2.2 System Architecture and Block Diagram 4

2.3 Client-Server Communication Sequence 6

2.4 Data Flow Diagram 9

3 Methodology 11

3.1 Introduction 11

3.2 Virtual Sensor 11

3.2.1 Configuration 11

3.2.2 Addressing a sample in either layout 12

3.2.3 The read-ahead window 12

3.2.4 Frames, blocks and the sensor's clock 13

3.2.5 Rejecting bad input, and the interface 14

3.3 Block Loader 14
3.4 Parallel Merge-Sort 15
3.5 Rolling Median Filter (Hampel Filter) 15
3.6 QNX IPC Mechanisms 15
3.6.1 Message Passing and IOV 15
3.6.2 QNX Pulses and Asynchronous Notification 15
3.6.3 Shared Memory 15

4 Results and Discussions **16**
4.1 Experimental Setup 16
4.2 Filter Quality and Outlier Removal (Width Sweep) 16
4.3 Processing Cost and Timing Analysis 16
4.4 Parallel Sorting Performance 16
4.5 System Overhead and IPC 16

5 Conclusion and Scope for Further Research **17**
5.1 Conclusion 17
5.2 Scope for Further Research 17

References **18**

List of Figures

2.1 System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem. 5

2.2 Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller. 7

2.3 Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter. 8

2.4 Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region. 10

3.1 How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped. 12

3.2 The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position. 13

List of Tables

3.1 Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column. 11

3.2 The sensor's interface, including the counters that report the recording and the sensor's own timeline. 14

Abbreviations

AWGN Additive White Gaussian Noise

BSP Board Support Package

FNV Fowler–Noll–Vo

IOV Input/Output Vector

IPC Inter-Process Communication

MAD Median Absolute Deviation

POSIX Portable Operating System Interface

QNX QNX Neutrino RTOS

RMSE Root Mean Square Error

RTOS Real-Time Operating System

SPM Single Particle Model

List of Symbols

σ Standard deviation (noise parameter)

w Filter width (window size in samples)

f_s Sampling frequency (Hz)

N Number of samples / block size

k Number of partitions / worker threads

Chapter 1

Introduction

1.1 Background and Motivation

Embedded systems used for real-time data acquisition often need to process large amounts of sensor data while meeting strict timing requirements. Multi-core processors help meet these demands by allowing computational tasks to run in parallel. However, making effective use of multiple cores requires proper task distribution and synchronization, especially in systems where predictable execution time and responsiveness are important.

Sensor data may also contain unwanted variations, such as sudden spikes, bursts of abnormal readings, or dropouts caused by electromagnetic interference, sensor faults, or changes in operating conditions. Identifying and removing these outliers is important for obtaining reliable results during statistical analysis. Combining this filtering process with data sorting increases the computational workload. In a Real-Time Operating System (RTOS), these operations must be handled efficiently without affecting the system's ability to respond to inter-process communication (IPC) requests and hardware interrupts.

1.2 Project Objectives

This project focuses on developing a real-time client-server system using the QNX RTOS and deploying it on a Raspberry Pi 4. The system is designed to transfer unsorted floating-point time-series data from a client to a server in blocks of at least 64 KB for processing. The server performs two main operations:

1. **Parallel Sorting:** A parallel Merge-Sort algorithm is implemented using multiple POSIX worker threads. Mutexes and condition variables are used to coordinate thread execution and synchronization during the sorting and merging stages.

2. **Rolling Median Filtering:** A robust rolling median filter (Hampel filter) is applied to identify and remove outliers from the time-series data. The filter uses local median and median absolute deviation (MAD) values to detect abnormal readings while preserving

the underlying signal.

After processing, the server notifies the client through a QNX pulse event containing the calculated median and a descriptor specifying the relevant memory offset. The processed arrays remain in shared memory, allowing the client to access the results without requiring another complete data transfer.

1.3 Workload and Dataset

The system is evaluated using a synthetic dataset generated offline with PyBaMM, a Python-based battery modelling package. The dataset contains battery measurements, including current, voltage, and temperature, representing a simulated discharge cycle. To evaluate the ability of the filtering algorithm to detect abnormal readings, controlled noise and outliers are introduced into the data.

The dataset includes the following types of corruption:

- **Gaussian Noise:** Additive white Gaussian noise is introduced into the measurements to simulate variations in sensor readings. Example noise levels include 2 mV for voltage and 8 mA for current.
- **Impulsive Outliers:** Outliers are introduced in the form of sudden spikes, consecutive bursts, and stuck readings. These outliers are introduced at random and occur at a rate of 1.5%. A ground-truth mask is also maintained to identify the injected outliers and evaluate the filtering performance.

This dataset provides a controlled environment for evaluating the sorting and filtering algorithms under different data conditions. It also allows the results to be compared against the known locations of the injected outliers.

1.4 Report Structure

The remainder of this report is organized into four chapters:

- **Chapter 2 (System Overview)** presents the system architecture, including the block diagram, data flow, and sequence of messages exchanged between the client and server.
- **Chapter 3 (Process Implementation)** explains the implementation of the parallel Merge-Sort algorithm, the rolling median filter, and the QNX IPC mechanisms used for communication and data sharing, including IOV transfers, pulses, and shared memory.
- **Chapter 4 (Results and Discussions)** evaluates the system's performance through thread-scaling experiments, filtering accuracy measurements, and execution-time analysis on the Raspberry Pi 4.
- **Chapter 5 (Conclusion)** summarizes the project outcomes and discusses possible improvements and directions for future work.

Chapter 2

System Overview

2.1 Introduction

This chapter provides a high-level overview of the system architecture. It describes the interaction between the client and server, the flow of data, and the communication mechanisms utilized within the QNX RTOS.

2.2 System Architecture and Block Diagram

The system is based on a client-server model running on QNX. The client reads data from a virtual sensor and offloads it in 64 KB blocks to the server. The server utilizes a multi-threaded approach to sort the data and apply a Hampel filter. The results are written to a shared memory region.

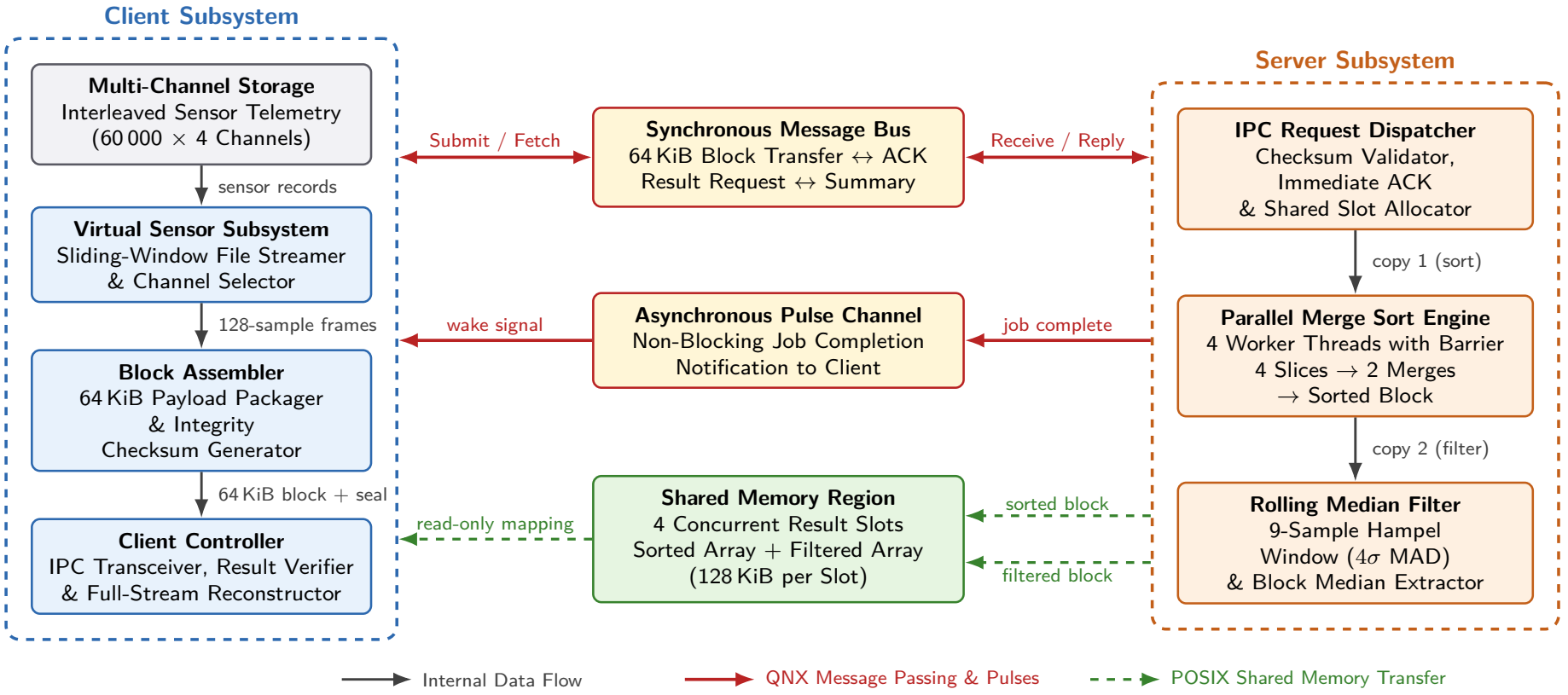


Figure 2.1: System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem.

2.3 Client-Server Communication Sequence

Communication between the two processes follows a strict sequence to ensure data integrity and asynchronous notification.

1. The client connects to the server using `name_open`.
2. The client sends a 64 KB block using an IOV message (`MsgSendv`).
3. The server validates the block, sends an acknowledgement, and queues the job.
4. Once processing is complete, the server sends a QNX pulse back to the client.
5. The client retrieves the result descriptor using a fetch message.

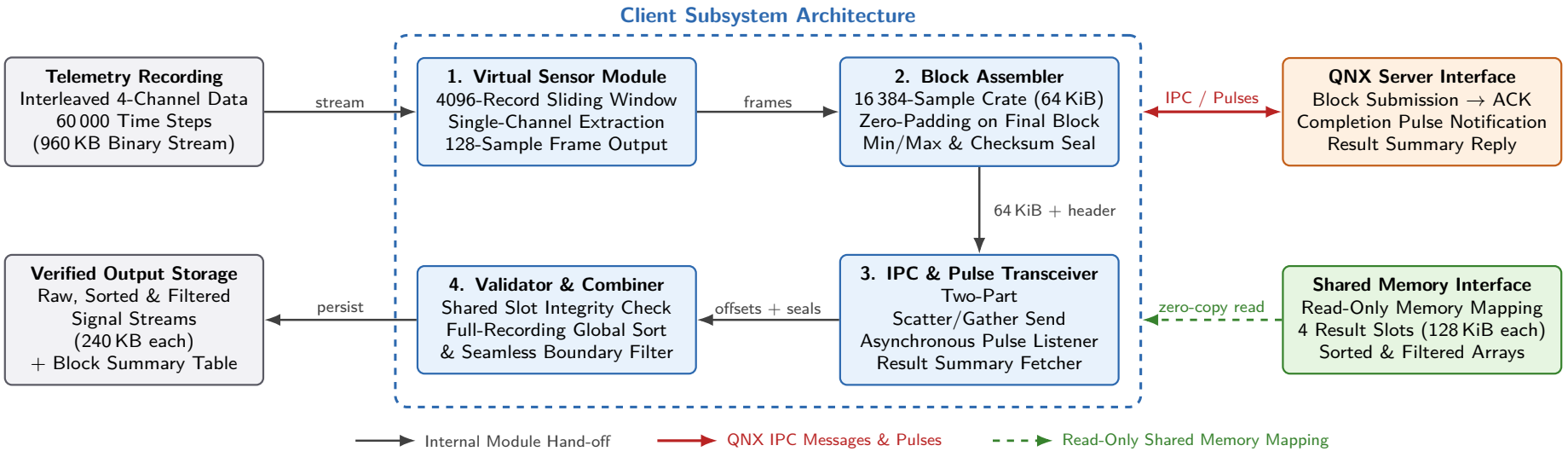


Figure 2.2: Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller.

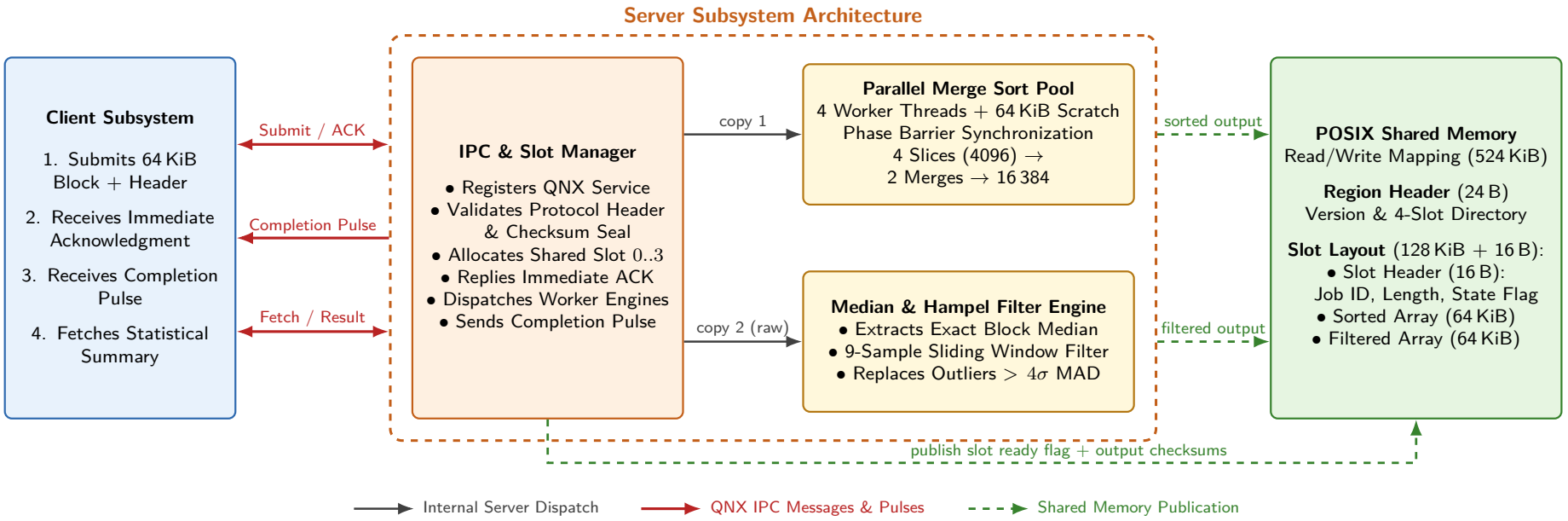


Figure 2.3: Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter.

2.4 Data Flow Diagram

The data starts as raw binary files on disk, is loaded by the client, transferred over the QNX message passing kernel, sorted and filtered, and finally written into a mapped shared memory region.

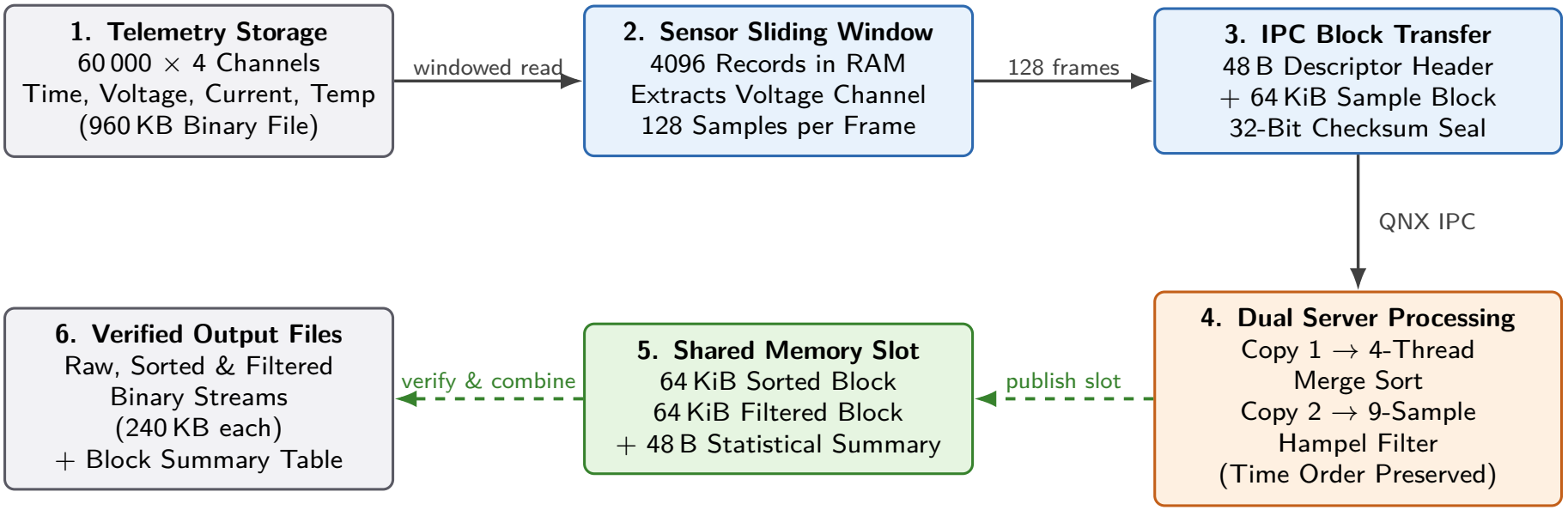


Figure 2.4: Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region.

Chapter 3

Methodology

3.1 Introduction

This chapter details the implementation of the core algorithms, the data structures, and the QNX-specific mechanisms used to build the service.

3.2 Virtual Sensor

There is no physical sensor on the bench, so the service is fed by a module that replays a recording with the behaviour of a live device. Values arrive one sample at a time, in frames, on a clock the module keeps itself rather than at the speed of the host. The purpose is not convenience: with the sensor behind this interface the client and the rest of the pipeline can be developed and tested exactly as they will run against hardware, and a real device can replace the module later without changing anything downstream.

3.2.1 Configuration

Everything the caller can choose sits in one structure, so the two file layouts are configuration rather than separate code paths:

Table 3.1: Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column.

Field	Type	Meaning	Default
`layout`	`int`	`SENSOR_LAYOUT_PLANAR`, one sample per float, or `SENSOR_LAYOUT_INTERLEAVED`, `stride` floats per record	planar
`offset`	`int`	interleaved only: which float of a record this sensor delivers	0
`stride`	`int`	interleaved only: floats per record	1
`frame_samples`	`size_t`	samples a single read may deliver	128
`rate_hz`	`uint32_t`	virtual rate used for timestamps and pacing	100
`loop`	`int`	restart at the end of the recording, as a live sensor would	0

3.2.2 Addressing a sample in either layout

Both layouts are headerless little-endian `float32`, and both are read by one expression, where `next` is the sample the sensor is about to deliver:

$$\texttt{value} = \texttt{window}\big[(\texttt{next} - \texttt{window_first}) \times \texttt{stride} + \texttt{offset}\big] \tag{3.1}$$

A planar file is `stride` 1 and `offset` 0, so sample i is float i. An interleaved file is four floats per record — time, voltage, current, temperature — so sample i of the voltage channel is float $4i + 1$: record 1 is float 5, byte 20 of the file. The three unused floats of each record are read from disk and discarded; a channel other than the selected one is never returned. Figure 3.1 shows both cases.

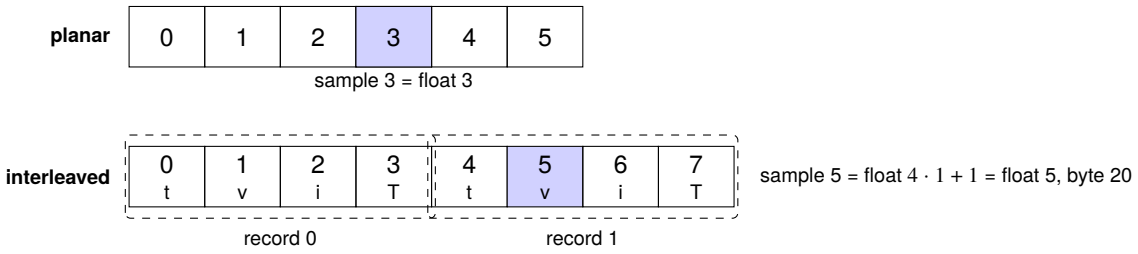


Figure 3.1: How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped.

3.2.3 The read-ahead window

The recording is not loaded whole. Reads pass through a sliding window of `window_samples` records — 4096 by default, widened when the caller's frame is larger — which is refilled as the reader advances. In the real pipeline the block loader asks for block-sized frames, so the window becomes 16 384 records: 64 KiB of a planar file or 256 KiB of an interleaved one, in one call to `fread`. A caller reading one sample at a time therefore sees exactly the same values as a caller reading whole blocks, a property the tests use as an independent check.

The refill condition is the three-part test in Listing 3.1: the window is empty, the reader was rewound behind it, or it has walked past the end. The loop applies the test before every sample, so no case can read stale data.

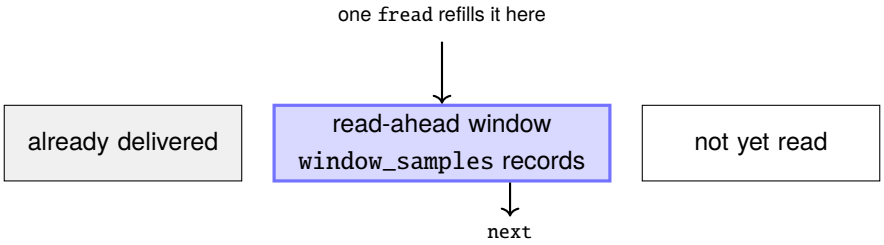


Figure 3.2: The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position.

Listing 3.1: The read loop of `sensor_read()`, trimmed of its error returns. A device read delivers at most one frame; aggregating frames into a block is the caller's job.

```
while (n < cap) {
    if (s->next >= s->total) {
        if (!s->cfg.loop)
            break; /* end of the recording */
        s->next = 0; /* a live sensor never stops */
    }

    if (s->window_count == 0 || /* nothing read yet */
        s->next < s->window_first || /* rewound behind the window */
        s->next >= s->window_first + s->window_count)
        sensor_fill(s, s->next); /* refill at the new position */

    offset = (s->next - s->window_first) * s->cfg.stride + s->cfg.offset;
    dst[n++] = s->window[offset]; /* the one channel that was asked for */
    s->next++;
    s->delivered++;
}
```

3.2.4 Frames, blocks and the sensor's clock

A device read returns a frame, not an arbitrary amount of data: `sensor_read` delivers at most `frame_samples` samples, and `sensor_read_frame` delivers exactly one frame except at the end of a recording. Assembling a 64 KiB payload is the client's concern, which is what `sensor_read_block` does: it calls `sensor_read` until the block is full, so the aggregation sits on the client side of the boundary rather than inside the device.

The clock is arithmetic rather than a system timer. The timestamp of the next sample is

`sensor_index` $\times 10^9 /$ `rate_hz` nanoseconds, computed from a counter of samples delivered, so after 600 samples at 100 Hz it is exactly 6 s however fast the host runs; in loop mode the counter keeps advancing across the wrap, as a sensor that never stops would. `sensor_read_paced` adds a wait after each read so the stream holds the configured rate. That wait and the monotonic clock behind it are the only platform-specific code in the project — `clock_gettime` and `nanosleep` on QNX, with one shim for the host build — and its resolution is a millisecond, so pacing is meaningful up to roughly 500 Hz.

3.2.5 Rejecting bad input, and the interface

Nothing is assumed about the file. A missing path, an empty file, a length that is not a whole number of `float32` values or of records, an offset outside the record, a stride below one, an unknown layout, a frame or rate out of range — each returns a null handle or an error code and prints a sentence naming the problem instead of reading nonsense. Reads follow the same discipline: the number of samples delivered, 0 at the end of a recording, -1 for an error.

Table 3.2: The sensor's interface, including the counters that report the recording and the sensor's own timeline.

Function	Purpose
`sensor_total_samples(s)`	number of samples in the recording
`sensor_remaining(s)`	samples left before the end
`sensor_index(s)`	samples delivered since open or reset
`sensor_open(path, cfg)`	open a recording; a null configuration selects the defaults
`sensor_read(s, dst, max)`	deliver at most one frame
`sensor_read_frame(s, dst)`	deliver exactly one frame
`sensor_read_block(s, dst, n)`	aggregate frames into a block of n samples
`sensor_read_paced(s, dst, max)`	read, then wait for the configured rate
`sensor_timestamp_ns(s)`	the next sample's time on the sensor's clock
`sensor_reset(s)`	rewind and restart the timeline
`sensor_close(s)`	release the handle

3.3 Block Loader

The block loader (`block_loader.c`) aggregates these samples into exact 64 KB blocks (`PROTO_BLOCK_SAMPLES` of 16384 float32 values).

3.4 Parallel Merge-Sort

The server sorts each block using a parallel Merge-Sort algorithm. The sort is partitioned across multiple POSIX worker threads. A mutex and two condition variables (`work_ready`, `phase_done`) are used to synchronize the phases, ensuring that merge operations only begin after all preceding partition tasks are complete.

3.5 Rolling Median Filter (Hampel Filter)

To remove outliers, the server applies a Hampel-style rolling median filter. The filter calculates the median and Median Absolute Deviation (MAD) of a sliding window. If a sample deviates from the median by more than a configurable threshold (4.0 sigma), it is replaced. This preserves legitimate signal edges while removing sudden spikes.

3.6 QNX IPC Mechanisms

3.6.1 Message Passing and IOV

Large data blocks are transferred using `MsgSendv` (Input/Output Vectors) to avoid copying the 64 KB buffer into a single contiguous message.

3.6.2 QNX Pulses and Asynchronous Notification

Upon completion, the server sends a pulse. Because a pulse value is limited to 32 bits, the float representation of the calculated median is bit-cast directly into the pulse payload. The pulse code carries the Job ID.

3.6.3 Shared Memory

To avoid sending the large processed arrays back over IPC, the server maps a named shared memory region (`/dev/shmem/sortmed-results`). The server writes the sorted and filtered arrays directly into this region, and the client maps it read-only to retrieve the results.

Chapter 4

Results and Discussions

4.1 Experimental Setup

This section describes the environment and the dataset used for the performance evaluations on the Raspberry Pi 4 board.

4.2 Filter Quality and Outlier Removal (Width Sweep)

This section presents the quality of the Hampel filter across different window widths against the ground-truth mask.

[Insert Figure: width_quality.png Here]

[Insert Figure: error_map.png Here]

4.3 Processing Cost and Timing Analysis

This section discusses the execution time per 64 KB block and how it scales with the filter width.

[Insert Figure: width_cost.png Here]

[Insert Figure: width_tradeoff.png Here]

4.4 Parallel Sorting Performance

This section briefly evaluates the execution time and speedup achieved by the parallel merge-sort on the four cores of the Raspberry Pi 4.

4.5 System Overhead and IPC

This section discusses the latency introduced by the QNX IPC mechanisms (message passing and shared memory).

Chapter 5

Conclusion and Scope for Further Research

5.1 Conclusion

The project successfully implemented a real-time client-server architecture on QNX. The system demonstrated effective parallel sorting using POSIX threads and robust outlier removal using a Hampel filter. The use of QNX pulses and shared memory provided an efficient, low-latency method for data transfer and asynchronous notification.

5.2 Scope for Further Research

Future work could include:

- Integrating a live hardware sensor (e.g., via I2C or SPI) instead of a virtual file-based sensor.
- Implementing dynamic thread allocation based on the current system load.
- Exploring alternative sorting algorithms (e.g., Radix Sort) for floating-point data.
- Adding a graphical user interface (GUI) for live monitoring of the metrics.

References

[1] QNX Software Systems, “QNX Neutrino RTOS Programmer’s Guide,” BlackBerry Limited, 2023.

[2] V. Sulzer, S. G. Marquis, R. Timms, M. Robinson, and D. A. Howey, “Python Battery Mathematical Modelling (PyBaMM),” *Journal of Open Research Software*, vol. 9, no. 1, 2021.

[3] F. R. Hampel, “The influence curve and its role in robust estimation,” *Journal of the American Statistical Association*, vol. 69, no. 346, pp. 383–393, 1974.